HOST:

[SMILE] Welcome to Episode 3 of the BIC AI Podcast.

Before we talk about AI, let's start with something very normal: a friends' road trip.

Imagine two groups going to the same hill station.

Same type of car.
Similar driving experience.
Same destination.

In the first group, planning is very simple.

Someone says, "Bro, Google Maps is there. Let's start. We'll figure it out."

Very confident.

But nobody has checked the route properly. Nobody knows if there is a road closure. Nobody checked the weather. And the person who booked the hotel is saying, "One second, I think the confirmation is in my other email."

[SMILE]

Now, on a friends' trip, this is still okay.

Worst case, someone will make fun of you for the next five years.

Every reunion, same story:

"Remember that trip where we reached a destination, but our hotel booking reached some other destinaiton?"

[AUDIENCE LAUGHTER / PAUSE]

But in enterprise delivery, it is not that funny.

If we say, "AI is there, let's start, we'll figure it out," and there is no planning around the work, the client will not call it adventure.

They will call it rework.
They will call it missed dependency.
They will call it lack of control.

And then, unfortunately, the joke is on us.

[PAUSE]

Now imagine the second group.

Same car. Same driver. Same destination.

But the journey is organized better.

The route is agreed.
Responsibilities are clear.
Someone has the hotel details.
Someone is watching navigation.
There are planned stops.
And there is one simple rule:

If the journey becomes unsafe, they stop and reassess.

[AUDIENCE] Which journey would you trust more?

The car?
The driver?
Or the way the journey is organized?

That is the idea we are exploring today.

Capability matters. But capability without a well-designed environment becomes guesswork.

In AI also, the model may be powerful.

Claude can reason.
Codex can code.
ChatGPT can explain.
Cursor can assist inside development work.

But in real team delivery, we need more than a powerful model.

We need a way to organize the work around it.

[EMPHASIZE] That organized environment is what we are calling the harness.

Harness Engineering is the discipline of designing that environment deliberately, so AI-assisted work becomes easier to repeat, easier to review and safer to improve.

05:00–15:00 | Harness Engineering Explained

Host: Predeep

Senthil + Priya Research Conversation

HOST:
Senthil, Priya — both of you have been researching Harness Engineering.

But let's not start with a definition.

Let's start with something simple.

When AI gives us an answer, what makes us trust it?

Is it speed?
Is it confidence?
Or is it because we know the work followed the right process?

Senthil, why is "harness" becoming such an important word now?

SENTHIL:
That's a good place to start, Predeep.

Most teams already have AI pieces today.

We have Claude.
We have prompts.
Some teams have Skills.
Some are exploring MCP connections.
Some are trying coding agents.

So AI is not missing.

The issue is that these pieces are often separate.

One person has a good prompt.
Another person has created a Skill.
A teammate added a tool integration. (Someone else has connected a tool.)

Then the agent gives an output, and the team still has to ask:

Did it use the right context?
Did it touch the right files?
Did it miss a dependency?
Did it run the right checks?
Can someone review what actually happened?

That gap is where Harness Engineering becomes important.

A harness is the working environment around the AI.

It tells the AI what it should know.
It controls what the AI can access.
It defines what the AI is allowed to do.
It decides what must be checked before the work is accepted.

So the model may be Claude.

But the harness is what makes Claude useful in a real team workflow.

SENTHIL:
Let me take a simple example.

Suppose we ask Claude to help with a small application enhancement.

On paper, it sounds simple.

Add a field.
Update validation.
Adjust the test.

But in real delivery, even a small change can spread.

Maybe the UI changes.
Maybe the API contract changes.
Maybe a downstream data pipeline depends on that field.
Maybe a BDD scenario needs to be updated.

Now, if we only say, “Claude, make this change,” we are expecting the model to discover the full impact by itself.

A harness changes how the work starts.

It says:

Read the requirement.
Identify impacted modules.
Check the contract.
Update only the allowed files.
Run the required tests.
Produce evidence for review.

That is the difference.

A prompt asks for an answer.
A harness shapes the work.

SENTHIL:
This is why people say things like “prompting is dead.”

I think that line is useful for attention.

But we should not take it literally.

Prompting is not dead. We still need clear instructions.

But prompting alone is not enough ,when the work needs to be repeatable.

A prompt cannot enforce access control.
A prompt cannot guarantee that tests run.
A prompt cannot preserve decisions across multiple steps.
A prompt cannot prove that the output is ready for review.

So I would say it this way.

Prompting starts the task.
The harness controls the task.

SENTHIL:
There are three levels we can use to think about this.

The first is an **AI Assistant Harness**.

This is where we use Claude or ChatGPT for bounded knowledge work. For example, we may ask AI to review approved requirements and produce a gap checklist.

Here, the harness can be simple:

Approved inputs.
Reusable instructions.
Fixed output format.
Human review.

The second is a **Coding Agent Harness**.

This is where tools like Claude Code, Codex or Cursor come in.

Now the agent is not just writing text. It may inspect files. It may suggest code changes. It may run tests. It may even create a pull request.

So the harness needs stronger controls:

Repository instructions.
Scoped access.
Test hooks.
Isolated workspaces.
Review gates.

The third is an **Agentic System Harness**.

That is where teams build a full system around orchestration, memory, monitoring and governance.

But for this episode, we should stay mostly with the first two levels.

That is where most teams can start immediately.

We don’t need every team to build a full AI platform. But every team can build a better harness around one recurring task.

HOST:
That makes sense.

So harness is not another tool name.

It is the operating setup around the AI.

Priya, if a team wants to check whether their harness is complete, what should they look for?

PRIYA:
A simple way is to ask six questions.

The research term for this is **ETCSLV**.

But let's treat it as a checklist.

Execution.
How does the task start?
How does it stop?
What happens if it fails?

Tools.
What systems can the AI use?
What files can it touch?
What actions are allowed?

Context.
What information does the AI need before it starts?

State.
What decisions should survive between steps?

Lifecycle.
Who owns authentication?
Who owns escalation?
Who handles cost, timeout and handoff?

Verification.
What proves that the work is actually correct?

That is all ETCSLV is doing.

It helps us avoid one dangerous assumption:

"The AI gave an answer, so the task is done."

PRIYA:
Let's make this real with three quick team examples.

First, application development.

A coding agent may update the code. But the harness asks:

Did it read the Jira story?
Did it identify impacted files?
Did it run the right tests?
Did it show what changed?

Second, data engineering.

A data pipeline may run successfully. Row counts may match.

But the report can still be wrong if one source column changed meaning.

So a data harness asks:

What is the approved data contract?
What reconciliation rules must pass?
What business validation is required before sign-off?

Third, production support.

An AI assistant can summarize logs quickly. That is useful.

But should it restart a service?
Should it raise an incident?
Should it suggest rollback?

Those actions need a controlled path.

The harness can say:

Collect logs.
Classify severity.
Check recent deployments.
Compare with the runbook.
Ask for human approval before risky action.

So across application, data and support, the idea is the same.

The AI should not just respond.
It should follow our way of working.

HOST:
That connects well.

Senthil, you were also looking at Databricks and Omnigent.

Let’s bring that in only as a proof point.

Why should our audience care about Databricks in a Harness Engineering discussion?

SENTHIL:
Because Databricks tested this idea on real engineering work.

Not a toy benchmark.
Not a demo task.
Not an LLM judging another LLM.

They used real merged pull requests from their own multi-million-line codebase. The codebase covered more than ten languages, and the agents were evaluated using actual tests.

And the important finding was this:

The model alone did not decide the outcome.

The harness around the model changed the cost, the context usage and the completion result.

So the lesson is not, “Use this Databricks product.”

The lesson is:

If we want serious AI engineering, we must measure the full agent setup — not just the model name.

PRIYA:
Exactly.

And one number makes this very clear.

Databricks reported that changing the harness around the same model changed cost per task by more than two times in some comparisons, while quality stayed broadly comparable.

That is a strong signal for our teams.

We often ask, “Which model are we using?”

But the better question is:

How much context are we sending?
How much rework is happening?
What checks are running?
How do we know the task is complete?

In simple words:

Cost per token is not the real metric.
Cost per accepted task is.

So when we build a Claude workflow, we should not celebrate only because Claude produced an output.

We should ask whether the harness helped us finish the work with less rework, better evidence and fewer unnecessary steps.

PRIYA:
This is where the idea of a **meta-harness** comes in.

Different teams may use different AI workbenches.

One team may use Claude Code.
Another may use AI for testing.
Another may use a data-reconciliation assistant.
Another may use AI for production triage.

A meta-harness sits above these setups and asks:

Can we apply common policies?
Can we keep common approvals?
Can we maintain one audit trail?
Can we change the model or agent tool without rebuilding the full workflow?

That is where Omnigent becomes a useful Databricks example.

But the important point is not the brand name.

The important point is portability.

The team's requirements, permissions, checks and evidence should not be trapped inside one tool. They should remain reusable as the AI landscape changes.

SENTHIL:
That is also the leadership angle.

If every team builds separately, governance becomes scattered.

One team has one review process.
Another team has another evidence format.
A third team has a different approval rule.

A meta-harness mindset helps us separate two things:

What should be common across teams?
And what should remain specific to each team's workflow?

That is a practical way to think about a Harness Catalog.

Not a catalog of prompts.

A catalog of reusable workflow patterns.

PRIYA:
There are two more emerging ideas worth mentioning, but let's keep them short.

The first is **AutoHarness**.

AutoHarness is about reducing the manual effort needed to create constraints.

Some agent actions are safe, like reading logs.

Some are risky, like writing to production or restarting a service.

An AutoHarness-style approach tries to generate constraint logic from the tool setup and the task environment.

But it is not magic.

In an enterprise, any generated constraint still needs review, testing, approval and monitoring.

The useful idea is this:

Over time, even harness building can become more automated.

PRIYA:
The second idea is **Natural-Language Harnesses**.

This solves a different problem.

Many domain experts understand the workflow very well, but they do not want to edit backend orchestration code.

For example, a production-support lead may want a rule like this:

If the alert is severity two or higher, check recent deployments.
Compare with the runbook.
Summarize customer impact.
Ask for approval before restart.

That rule is easy to understand in natural language.

A Natural-Language Harness approach makes workflow control easier for domain experts to inspect and shape.

But it still needs a runtime underneath.

It still needs tools, permissions, versioning, tests and audit.

So it is not:

“Write anything in English and it becomes safe.”

It is:

“Make the control logic visible to the people who own the process.”

SENTHIL:
So if we simplify everything, we can remember it this way.

A meta-harness coordinates multiple harnesses.
AutoHarness helps generate or improve parts of the harness.
Natural-Language Harnesses make control logic readable for domain experts.

Three different problems.
Three different solutions.

For our teams, the immediate action is not to build all three tomorrow.

The immediate action is to pick one recurring task and design a small harness around it.

Start with one workflow.

Define the context.
Define the tools.
Define the state.
Define the lifecycle.
Define the verification.

Then measure whether the team gets better outcomes.

PRIYA:
And the success measure should be practical.

Did we reduce repeated explanation?
Did we reduce missed dependencies?
Did we reduce review effort?
Did we improve evidence?
Did we make the workflow safer and easier to repeat?

If yes, the harness is creating value.

AI value does not come only from better answers.

It comes from better-controlled work.

HOST:
That is the perfect bridge to our expert conversation.

We have heard the research view.

Harness as the environment.
ETCSLV as the checklist.
Databricks as evidence that harness choice changes outcomes.
Meta-harness, AutoHarness and Natural-Language Harnesses as future directions.

Now let’s move from research to real implementation.

Our expert has worked on application-development workflows, rework and feature enhancements.

He also brings experience with SDD, BDD, BMAD and a reverse-engineering agentic system where Jira, agents, Skills and prompts come together.

So the next question is simple:

What happens when a team builds the harness before asking the AI to work?